

Stack & Queue Part 1*Striver SDE Sheet - Precise Revision Notes*

Use this document for quick revision before interviews. Each problem includes the main intuition, Striver-style approach, and time/space complexity.

Quick Pattern Map

Problem Type	Best Structure	Remember
LIFO operations	Stack	Last inserted comes out first
FIFO operations	Queue	First inserted comes out first
Next greater / smaller	Monotonic Stack	Remove useless elements
Balanced brackets	Stack	Latest opened bracket closes first

1. Implement Stack using Array

Problem: Design stack operations push, pop, top, size, and isEmpty.

Core idea: Use an array and a top pointer. top = -1 means stack is empty.

Approach

1. For push, check overflow, increment top, and place the value.
2. For pop, check underflow, return/remove arr[top], then decrement top.
3. For top, return arr[top] if stack is not empty.
4. For size, return top + 1.

Important Notes

- Stack follows LIFO: Last In, First Out.
- Always handle empty stack before pop/top.

Complexity

Push / Pop / Top: O(1) | Space: O(n)

2. Implement Queue using Array

Problem: Design queue operations push, pop, front, size, and isEmpty.

Core idea: Use front and rear pointers. Prefer circular queue to reuse deleted spaces.

Approach

1. For push, move rear using circular formula and insert value.
2. For pop, remove front value and move front using circular formula.
3. If queue becomes empty, reset front and rear if needed.
4. Maintain current size to easily check full/empty.

Important Notes

- Queue follows FIFO: First In, First Out.

- Circular queue avoids wastage of array space.

Key Formula / Template

$\text{rear} = (\text{rear} + 1) \% \text{capacity}$

$\text{front} = (\text{front} + 1) \% \text{capacity}$

Complexity

Push / Pop / Front: $O(1)$ | Space: $O(n)$

3. Implement Stack using Queue

Problem: Implement stack behavior using queue operations.

Core idea: Use one queue. After pushing a new element, rotate previous elements behind it so the newest element becomes the front.

Approach

1. Push x into queue.
2. Rotate size - 1 elements: remove from front and add to rear.
3. Now queue front represents stack top.
4. Pop and top are performed from queue front.

Important Notes

- This converts FIFO order into LIFO behavior.
- Example after push 1,2,3: queue becomes [3,2,1].

Complexity

Push: $O(n)$ | Pop / Top: $O(1)$ | Space: $O(n)$

4. Implement Queue using Stack

Problem: Implement queue behavior using stack operations.

Core idea: Use two stacks. Push into input stack. For pop/peek, move elements to output stack only when output is empty.

Approach

1. Push directly into stack s1.
2. For pop/peek, if s2 is empty, move all elements from s1 to s2.
3. Top of s2 is the front of the queue.
4. Do not move elements repeatedly; move only when s2 is empty.

Important Notes

- Moving from s1 to s2 reverses order and gives FIFO.
- Each element moves at most once from s1 to s2.

Complexity

Push: $O(1)$ | Pop / Peek: Amortized $O(1)$ | Space: $O(n)$

5. Valid Parentheses

Problem: Check whether a bracket string is balanced.

Core idea: Push opening brackets. For each closing bracket, the stack top must be its matching opening bracket.

Approach

1. Traverse each character.
2. If it is an opening bracket, push it.
3. If it is a closing bracket, stack must not be empty and top must match.
4. After traversal, stack must be empty.

Important Notes

- A closing bracket should match the latest unmatched opening bracket.
- Mismatch or leftover opening brackets means invalid.

Key Formula / Template

) matches (

} matches {

] matches [

Complexity

Time: $O(n)$ | **Space:** $O(n)$

6. Next Greater Element

Problem: For every element, find the next greater element on the right.

Core idea: Traverse from right to left using a decreasing monotonic stack of useful greater candidates.

Approach

1. Start from the last element.
2. While stack is not empty and top $\leq$ current, pop it.
3. If stack is empty, answer is -1; otherwise top is next greater.
4. Push current element into stack.

Important Notes

- Smaller/equal elements are useless for the current and future left elements.
- Use indices if the problem needs positions.

Complexity

Brute force: $O(n^2)$ | **Optimal time:** $O(n)$ | **Space:** $O(n)$

7. Sort a Stack

Problem: Sort a stack using recursion.

Core idea: Pop the top, sort the remaining stack recursively, then insert the popped value at the correct sorted position.

Approach

1. If stack is empty, return.
2. Pop top element x.
3. Recursively sort the remaining stack.
4. Insert x into the sorted stack using another recursive function.

Important Notes

- insertSorted() places one value at the correct position.
- No extra data structure is required except recursion stack.

Complexity

Time: $O(n^2)$ | **Space:** $O(n)$ recursion stack

8. Next Smaller Element

Problem: For every element, find the next smaller element on the right.

Core idea: Traverse from right to left using an increasing monotonic stack of useful smaller candidates.

Approach

1. Start from the last element.
2. While stack is not empty and top $\geq$ current, pop it.
3. If stack is empty, answer is -1; otherwise top is next smaller.
4. Push current element into stack.

Important Notes

- This is the reverse condition of Next Greater Element.
- Greater/equal elements cannot be next smaller for current.

Complexity

Brute force: $O(n^2)$ | **Optimal time:** $O(n)$ | **Space:** $O(n)$

Final Templates

Monotonic Stack: Next Greater from Right

```
for (int i = n - 1; i >= 0; i--) { while (!st.isEmpty() && st.peek() <= arr[i]) st.pop(); ans[i] = st.isEmpty() ? -1 : st.peek(); st.push(arr[i]); }
```

Queue using Two Stacks Rule

push -> s1.push(x); pop/peek -> if s2 empty, move all s1 to s2; answer is s2.peek()/pop()